Student Course Management System

A Detailed College Project Report

Subject: Web Application Programming

Submitted By: __BINAY KUMAR CHAND

Student ID / Roll No.: __L.E.082BCT02_____

Program / Semester: __BCT-III/I_____

College / University: ____THAPATHALI CAMPUS_____

Submitted To: __GRIBAN ADHIKARI_____

Academic Year: 2082

TABLE OF CONTENTS

1. **Abstract** 2
2. **Introduction** 3
3. **Objectives** 4
4. **Scope of the Project** 4
5. **Technologies Used** 5
6. **System Architecture** 7
7. **Modules of the System** 7
8. **Database Design** 7
9. **API Design** 8
10. **Frontend Implementation** 8
11. **Backend Implementation** 9
12. **Authentication System** 9
13. **CRUD Operations** 10
14. **Testing** 10
15. **Advantages** 11
16. **Limitations** 11
17. **Future Enhancements** 11
18. **Conclusion** 12
19. **References** 13
20. **Appendix** 14

1. ABSTRACT

The Student Course Management System is a full-stack web application developed to streamline the management of students, courses, and enrollments in an academic environment. In traditional systems, managing such data manually or through disconnected tools often leads to inefficiency, redundancy, and data inconsistency. This project addresses those issues by providing a centralized and interactive platform that allows users to perform all essential operations in a structured and secure manner.

The system incorporates a modern web development stack, where the frontend is developed using React with Vite to ensure a responsive and dynamic user interface. The backend is implemented using Node.js and Express, which provides a robust environment for handling API requests and business logic. The application uses MySQL as the database to store structured data and maintain relationships between different entities such as students, courses, and enrollments.

Security is a critical component of this system. Therefore, authentication is implemented using JSON Web Tokens (JWT), which ensures that only authorized users can access protected resources. Once a user logs in, a token is generated and used for subsequent requests, maintaining session security without relying on traditional session storage.

Overall, the system demonstrates the integration of frontend, backend, and database technologies to build a real-world application capable of handling CRUD operations efficiently.

2. INTRODUCTION

With the rapid growth of digital transformation in education, institutions require efficient systems to manage academic data. Manual record-keeping is not only time-consuming but also prone to human errors, data duplication, and lack of accessibility. A web-based system provides a scalable and efficient solution to these problems.

The Student Course Management System is designed as a centralized platform where users can manage student information, course details, and enrollment records in a structured manner. The system allows users to interact with data through a user-friendly interface while ensuring that all operations are processed securely on the backend.

This project also serves as a practical implementation of full-stack web development concepts. It combines frontend technologies for user interaction, backend technologies for logic processing, and database systems for persistent storage. By integrating these components, the system demonstrates how modern applications are built and deployed.

The application is divided into multiple modules, each responsible for a specific functionality. These modules work together to provide a seamless experience for the user, ensuring that data flows correctly between the frontend and backend.

3. OBJECTIVES

The primary objective of this project is to develop a comprehensive system that can manage academic data effectively. The system is designed not only to perform basic operations but also to ensure security, scalability, and usability.

The specific objectives include:

The system aims to provide a secure authentication mechanism so that only authorized users can access the application. This is achieved through the implementation of JWT-based authentication, which ensures that user sessions are handled securely.

Another important objective is to implement complete CRUD operations for multiple entities. This includes managing students, courses, and enrollments. Each entity is designed to support creation, viewing, updating, and deletion of records.

The project also focuses on integrating frontend and backend systems. The frontend communicates with the backend through RESTful APIs, and the backend processes these requests and interacts with the database.

Additionally, the system is designed to demonstrate real-world application development using modern technologies such as React, Node.js, Express, and MySQL.

4. SCOPE OF THE PROJECT

The scope of this project is limited to managing student-related academic data within a controlled environment. It is designed primarily for educational purposes and small-scale implementations.

The system can be used by:

- Students or administrators to manage academic records

- Developers to understand full-stack architecture
- Institutions for basic course and enrollment tracking

However, the system does not include advanced features such as role-based access control, reporting dashboards, or large-scale deployment considerations. These can be added in future enhancements.

5. TECHNOLOGIES USED

The development of this system involves multiple technologies that work together to provide a complete solution.

The frontend is developed using React, which is a popular JavaScript library for building user interfaces. React allows developers to create reusable components and manage application state efficiently. Vite is used as the build tool, which provides faster development and optimized performance.

The backend is developed using Node.js, which provides a runtime environment for executing JavaScript on the server side. Express.js is used as the framework to build RESTful APIs and handle HTTP requests.

The database used in this project is MySQL, a relational database management system that allows structured storage of data. It supports relationships between tables, which is essential for linking students, courses, and enrollments.

Authentication is handled using JSON Web Tokens (JWT), which provide a secure way to transmit information between the client and server.

6. SYSTEM ARCHITECTURE

The system follows a client-server architecture, which is a widely used model in web development. In this architecture, the frontend acts as the client, and the backend acts as the server.

When a user interacts with the application, the frontend sends requests to the backend through HTTP calls. The backend processes these requests, performs necessary operations, and returns responses to the frontend.

The backend communicates with the MySQL database to store and retrieve data. This ensures that all information is stored persistently and can be accessed whenever needed.

The overall flow of the system can be described as follows:

The user interacts with the frontend interface, such as filling out a form or clicking a button. The frontend sends a request to the backend API. The backend processes the request, interacts with the database if necessary, and sends a response back to the frontend. The frontend then updates the user interface based on the response.

This architecture ensures separation of concerns, making the system easier to maintain and scale.

7. MODULES OF THE SYSTEM

The system is divided into several modules, each responsible for a specific functionality.

The authentication module handles user registration and login. It ensures that only authorized users can access the system. When a user logs in, a token is generated and used for authentication.

The student module allows users to manage student records. Users can add new students, view existing students, update their details, and delete records if necessary.

The course module is responsible for managing course information. It allows users to create new courses, update course details, and remove courses from the system.

The enrollment module connects students and courses. It allows users to assign students to specific courses and manage enrollment details such as semesters.

Each module is implemented both on the frontend and backend, ensuring proper communication and functionality.

8. DATABASE DESIGN

The database plays a crucial role in storing and managing data. In this project, MySQL is used to create a relational database with multiple tables.

The users table stores information about registered users, including their name, email, and password. Passwords are stored in a hashed format to ensure security.

The students table contains information about students, such as their name and email. Each student is assigned a unique identifier.

The courses table stores details about courses, including course names and identifiers.

The enrollments table establishes a relationship between students and courses. It includes references to student IDs and course IDs, along with additional information such as the semester.

This relational structure ensures data integrity and allows efficient querying of information.

9. API DESIGN

The backend provides RESTful APIs that allow the frontend to interact with the system.

The authentication APIs handle user registration and login. When a user registers, their information is stored in the database. During login, the system verifies credentials and generates a token.

The student APIs allow the frontend to perform CRUD operations on student data. These APIs handle requests such as retrieving all students, adding new students, updating existing records, and deleting students.

Similarly, course APIs manage course-related data, and enrollment APIs manage the relationships between students and courses.

Each API follows standard HTTP methods such as GET, POST, PUT, and DELETE, ensuring consistency and clarity.

---

10. FRONTEND IMPLEMENTATION

The frontend is designed to provide a user-friendly interface. It consists of multiple pages, each corresponding to a specific functionality.

The login and registration pages allow users to authenticate themselves. These pages include forms for entering user credentials and handling validation.

The dashboard provides an overview of the system and allows navigation to different modules.

The students page displays a list of students and provides options to add, edit, or delete records. Similarly, the courses page manages course information, and the enrollments page handles student-course relationships.

React Router is used for navigation between pages, and Axios is used for making API calls to the backend.

---

11. BACKEND IMPLEMENTATION

The backend is responsible for handling all server-side operations. It is built using Node.js and Express, which provide a flexible and efficient environment for building web applications.

The server initializes the application, sets up middleware, and defines routes for handling requests. Middleware functions are used for tasks such as authentication and data validation.

The backend connects to the MySQL database using a connection module, ensuring that all database operations are handled efficiently.

Each route corresponds to a specific functionality, such as authentication, student management, course management, or enrollment management.

---

12. AUTHENTICATION SYSTEM

Authentication is implemented using JWT, which provides a secure method for managing user sessions.

When a user logs in, the server generates a token that contains user information. This token is sent to the client and stored locally. For subsequent requests, the token is included in the request headers, allowing the server to verify the user's identity.

This approach eliminates the need for server-side session storage and improves scalability.

13. CRUD OPERATIONS

CRUD operations form the core functionality of the system. They allow users to create, read, update, and delete data.

In the student module, users can add new students, view all students, update student details, and remove students from the system.

In the course module, users can manage course information similarly.

The enrollment module allows users to assign students to courses, update enrollment details, and remove enrollments.

These operations ensure that users have full control over the data in the system.

14. TESTING

The system was tested manually to ensure that all functionalities work as expected. Each module was tested individually, and integration testing was performed to verify the interaction between modules.

Authentication was tested by registering and logging in with different users. CRUD operations were tested by performing all possible actions on students, courses, and enrollments.

Error handling was also tested to ensure that the system responds appropriately to invalid inputs.

15. ADVANTAGES

The system offers several advantages. It provides a centralized platform for managing academic data, reducing the need for manual record-keeping. The use of modern technologies ensures that the system is efficient and scalable.

The implementation of JWT authentication enhances security, ensuring that only authorized users can access the system. The modular design makes it easy to maintain and extend.

16. LIMITATIONS

Despite its advantages, the system has some limitations. It does not include role-based access control, meaning all users have similar privileges. The user interface is basic and can be improved for better user experience.

Additionally, the system does not include advanced features such as reporting, analytics, or notifications.

17. FUTURE ENHANCEMENTS

Several improvements can be made to enhance the system. These include adding role-based access control, implementing advanced dashboards with charts, and improving the user interface.

Other enhancements may include email notifications, mobile responsiveness, and integration with other systems.

18. CONCLUSION

he **Student Course Management System** successfully demonstrates the design and implementation of a complete full-stack web application by integrating frontend, backend, and database technologies into a unified system. Through this project, a practical understanding of how modern web applications function has been achieved, including how user interfaces communicate with servers and how data is stored, retrieved, and managed efficiently.

The system effectively implements core functionalities such as **user authentication, student management, course management, and enrollment handling**, all based on CRUD (Create, Read, Update, Delete) operations. The use of **JWT-based authentication** ensures secure access to protected routes, while the integration of a **MySQL relational database** demonstrates proper use of primary keys, foreign keys, and table relationships. This reflects the importance of structured database design in real-world applications.

From a development perspective, the project emphasizes the importance of **clean architecture and modular design**, where frontend (React), backend (Node.js with Express), and database layers operate independently yet cohesively. This separation of concerns improves maintainability, scalability, and readability of the code. Additionally, the use of tools such as **Git and GitHub** for version control highlights professional development practices and collaboration readiness.

The project also provides valuable hands-on experience in handling real-world challenges such as **API integration, error handling, data validation, and user interface design**. It demonstrates how different technologies work together to create a responsive and user-friendly application.

Furthermore, this system can be extended in the future by adding advanced features such as **role-based access control (admin/student), analytics dashboards, search and filtering options, and deployment to cloud platforms**. Therefore, it serves not only as a completed academic project but also as a strong foundation for future enhancements and professional development.

In conclusion, this project successfully fulfills its objectives and showcases the practical application of web development concepts, making it a valuable learning experience and a solid step toward building more complex software systems.

19. REFERENCES

Official Documentation

- React Official Documentation. Available at: https://react.dev
- Node.js Official Documentation. Available at: https://nodejs.org
- Express.js Official Documentation. Available at: https://expressjs.com
- MySQL Official Documentation. Available at: https://dev.mysql.com/doc/
- JSON Web Token (JWT) Documentation. Available at: https://jwt.io

Learning Resources

- Mozilla Developer Network (MDN) Web Docs – https://developer.mozilla.org
(Used for understanding JavaScript, HTML, and CSS concepts)
- W3Schools – https://www.w3schools.com
(Used for quick reference and syntax understanding)
- FreeCodeCamp – https://www.freecodecamp.org
(Used for learning full-stack development concepts)
- GeeksforGeeks – https://www.geeksforgeeks.org
(Used for database and programming explanations)

Tutorials & Guides

- Traversy Media (YouTube Channel)
(Used for understanding Node.js, Express, and React)

- Programming with Mosh (YouTube Channel)

(Used for backend and frontend concepts)

- CodeWithHarry (YouTube Channel)

(Used for practical understanding of web development)

Libraries & Tools Documentation

- Axios Documentation – https://axios-http.com
- Nodemon Documentation – https://nodemon.io
- bcryptjs Documentation – https://www.npmjs.com/package/bcryptjs
- cors Documentation – https://www.npmjs.com/package/cors
- dotenv Documentation – https://www.npmjs.com/package/dotenv

20. APPENDIX

The appendix section provides additional supporting information related to the project, including file structure, database schema, important code explanations, and execution steps.

20.1 Project Folder Structure

The project is organized into two main parts: frontend and backend.

```
fullstack-student-course-app/
│
├── backend/
│   ├── server.js
│   ├── db.js
│   ├── routes/
│   │   ├── auth.js
│   │   ├── students.js
│   │   ├── courses.js
│   │   ├── enrollments.js
│   ├── middleware/
```

```
│   │   └── auth.js
│   └── schema.sql
│
├── frontend/
│   ├── src/
│   │   ├── main.jsx
│   │   ├── App.jsx
│   │   ├── api.js
│   │   ├── components/
│   │   │   └── Navbar.jsx
│   │   ├── pages/
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Students.jsx
│   │   │   ├── Courses.jsx
│   │   │   ├── Enrollments.jsx
│   │   └── styles.css
│
└── README.md
```

20.2 Database Schema (SQL Tables)

The database consists of four main tables:

Users Table

Stores registered user information.

```
  CREATE TABLE users (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(100) UNIQUE,
```

```
  password VARCHAR(255)
);
```

Students Table

Stores student records.

```
  CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100)
);
```

Courses Table

Stores course information.

```
  CREATE TABLE courses (
  id INT AUTO_INCREMENT PRIMARY KEY,
  course_name VARCHAR(100)
);
```

Enrollments Table

Links students and courses.

```
  CREATE TABLE enrollments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT,
  course_id INT,
  semester VARCHAR(50),
  FOREIGN KEY (student_id) REFERENCES students(id),
  FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

20.3 Steps to Run the Project

To execute the project successfully, the following steps must be followed:

Backend Setup

First, navigate to the backend folder:

```
cd backend
```

Then install dependencies:

```
npm install
```

Run the backend server:

```
npm start
```

Frontend Setup

Open a new terminal and navigate to frontend:

```
cd frontend
```

Install dependencies:

```
npm install
```

Run frontend:

```
npm run dev
```

20.4 API Testing (Example)

Example of login request:

```
POST /login
{
  "email": "test@gmail.com",
  "password": "123456"
}
```

Example response:

```
{
  "token": "jwt_token_here"
}
```

20.5 Important Code Explanation

server.js

This file initializes the Express application, sets middleware such as CORS and JSON parsing, and connects all routes. It acts as the entry point of the backend.

db.js

This file establishes a connection with the MySQL database using connection credentials. It ensures that queries can be executed from different routes.

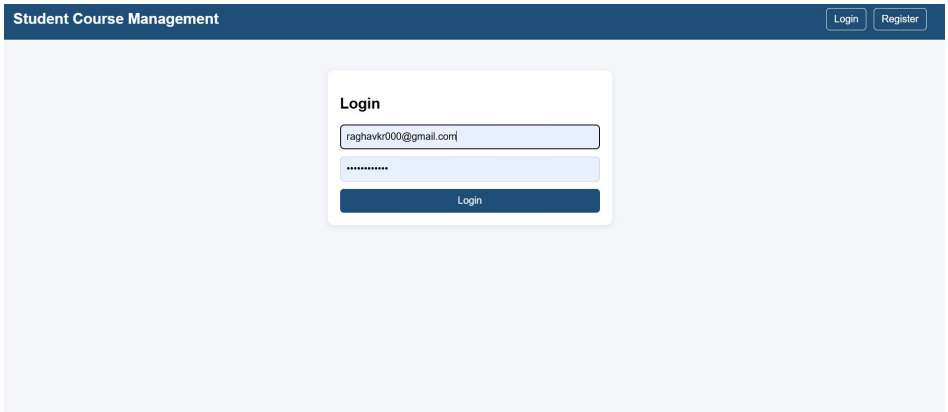
auth.js

Handles user registration and login functionality. Passwords are hashed before storing, and tokens are generated during login.

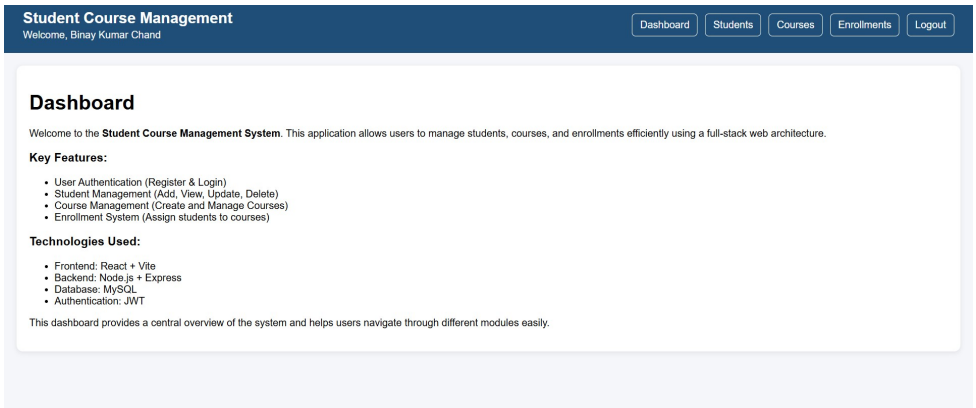
middleware/auth.js

This middleware verifies JWT tokens to protect secure routes. If the token is invalid, access is denied.

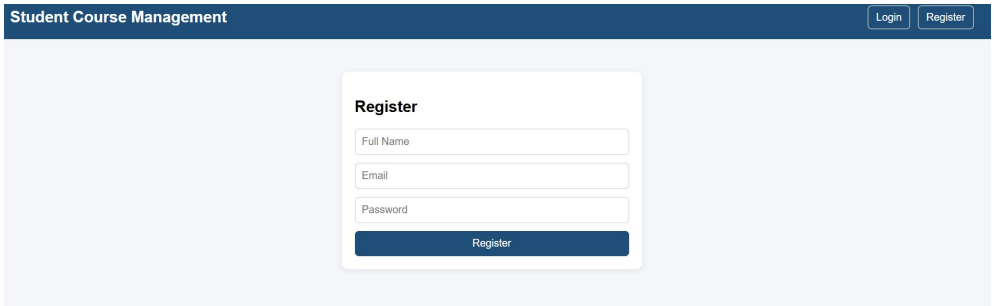
20.6 Screenshots Section



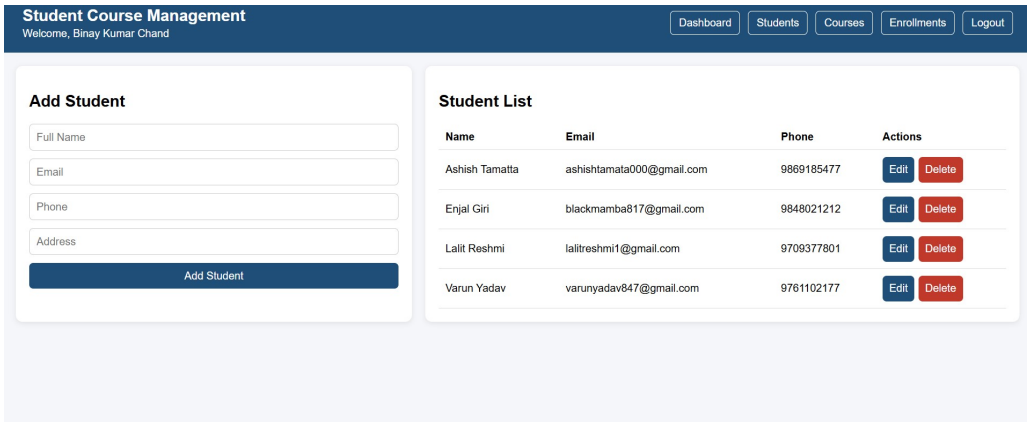
Login Page



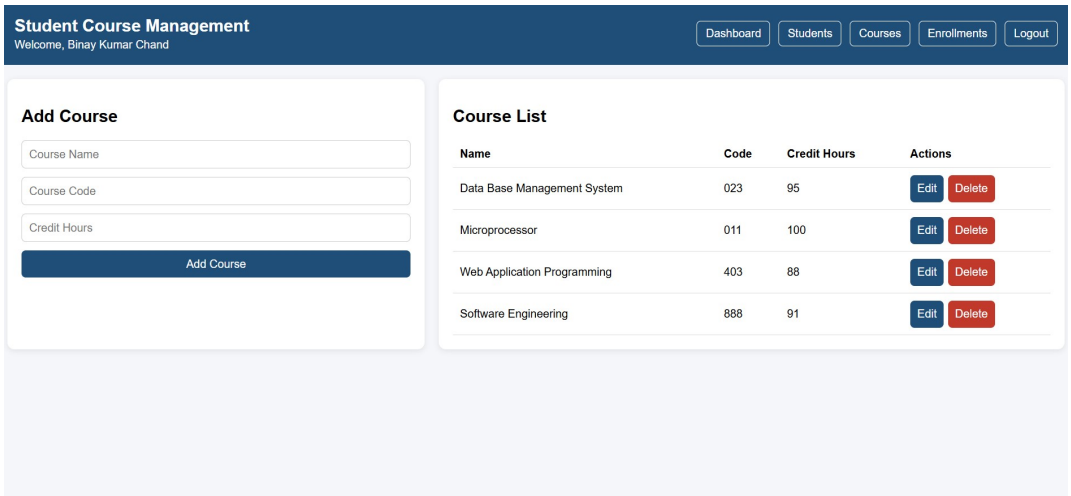
Dashboard Page



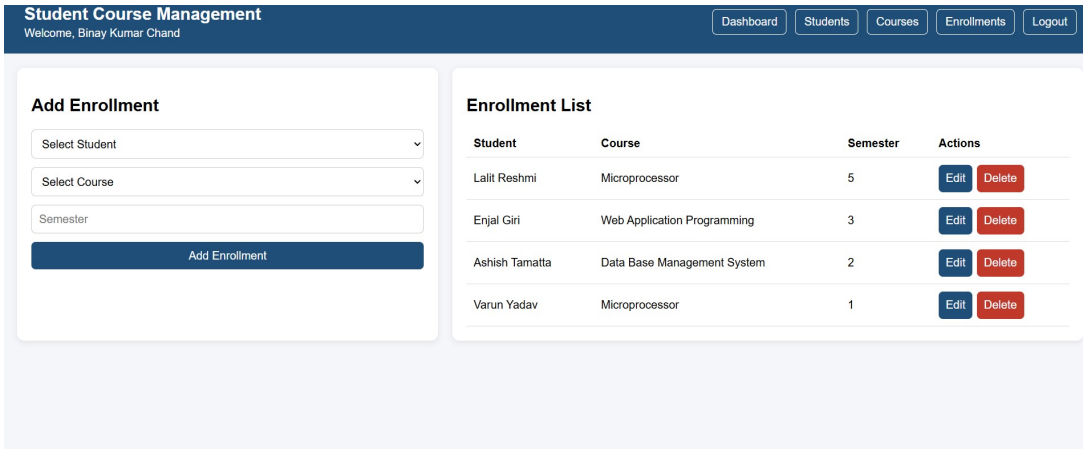
Register Page



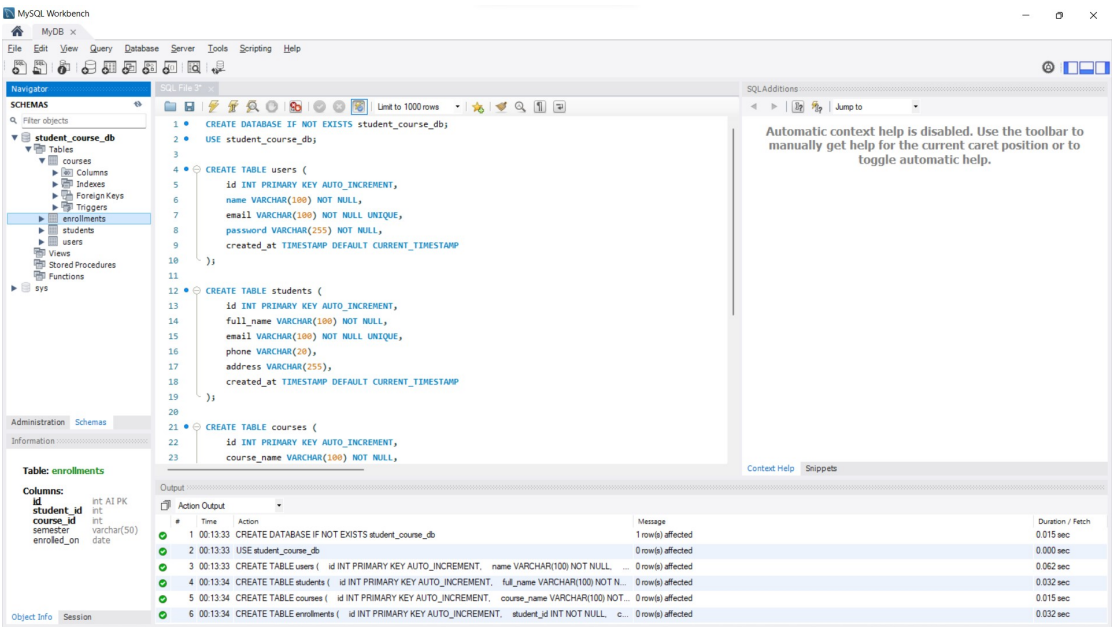
Student CRUD



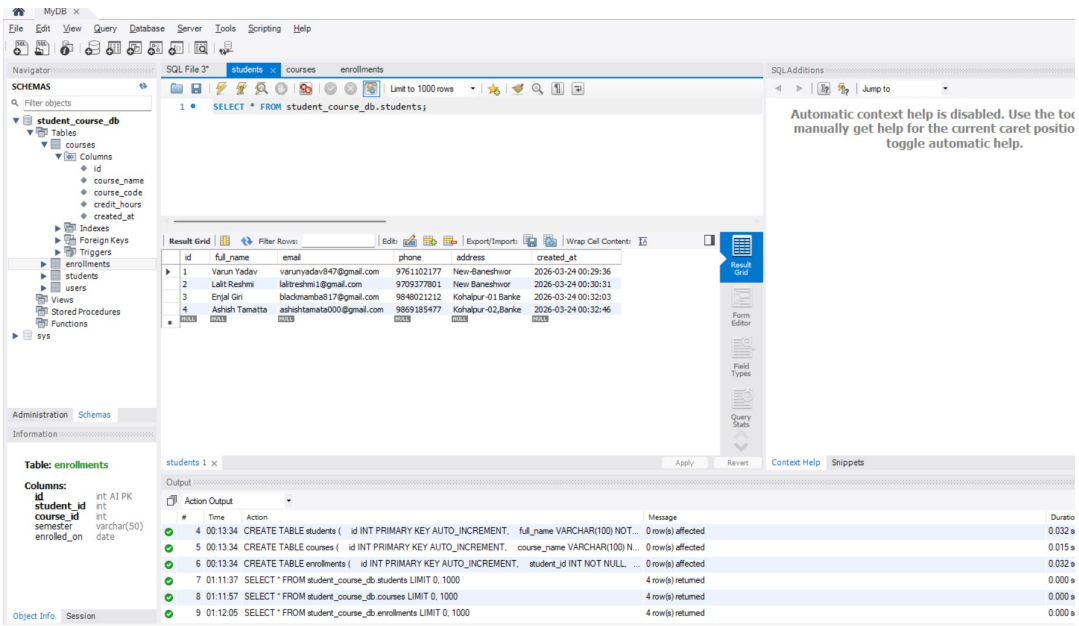
Course CRUD



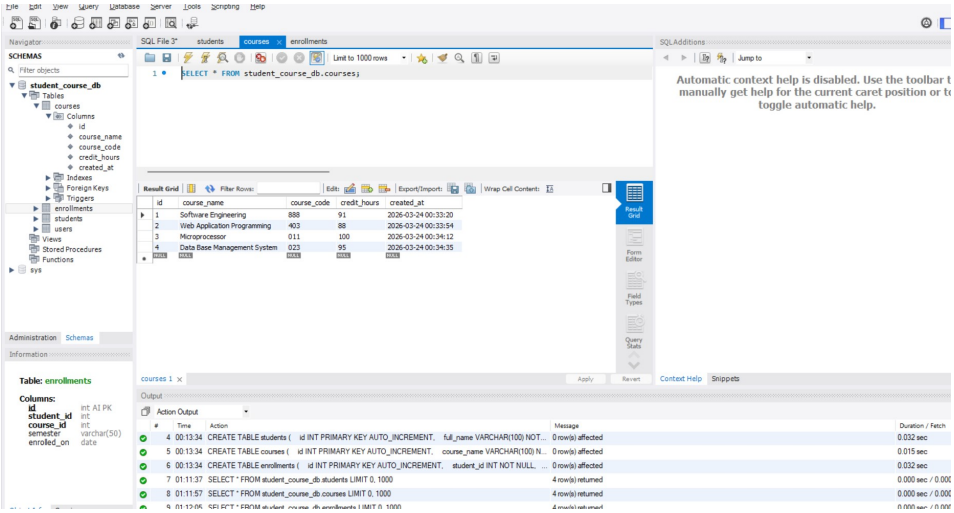
Enrollment CRUD



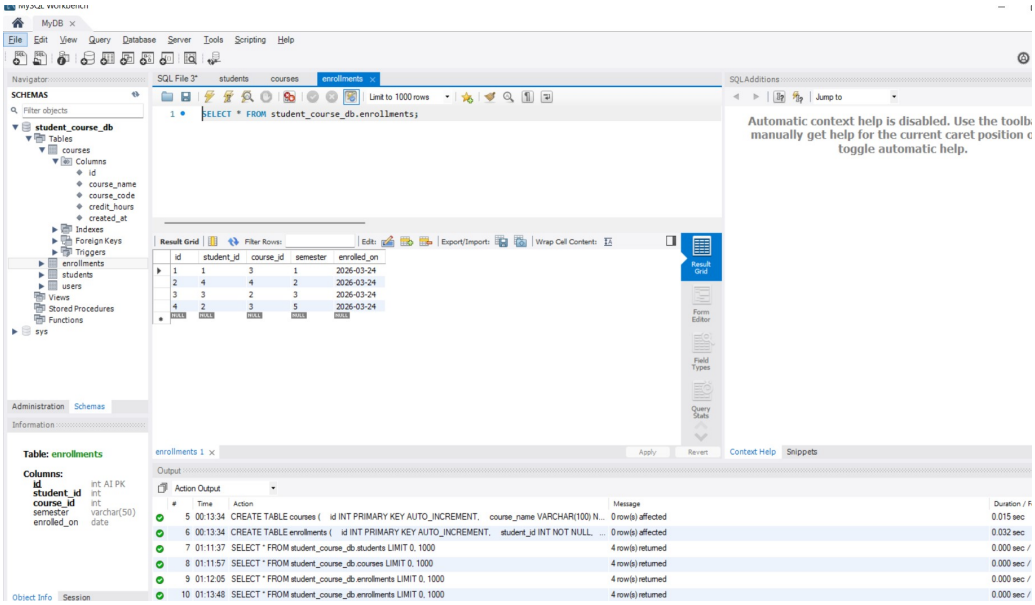
DATABASE Tables



Student database

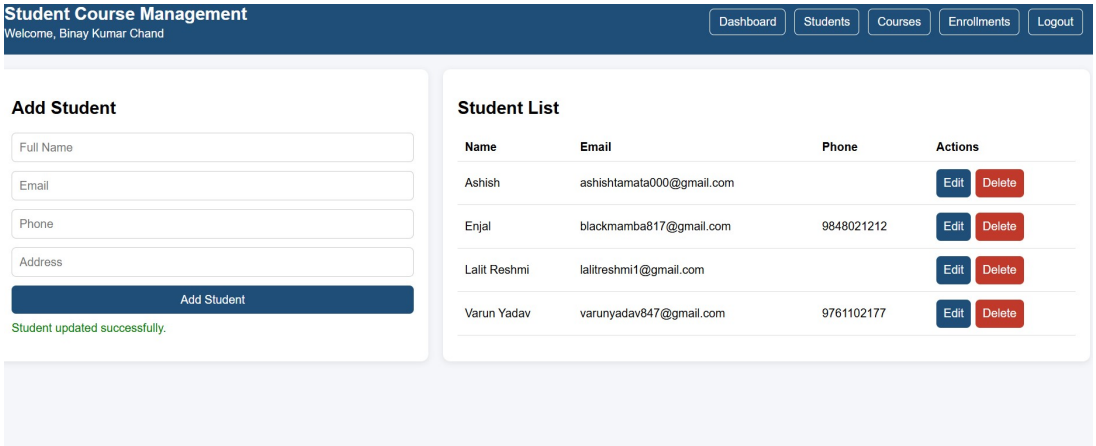


Course database

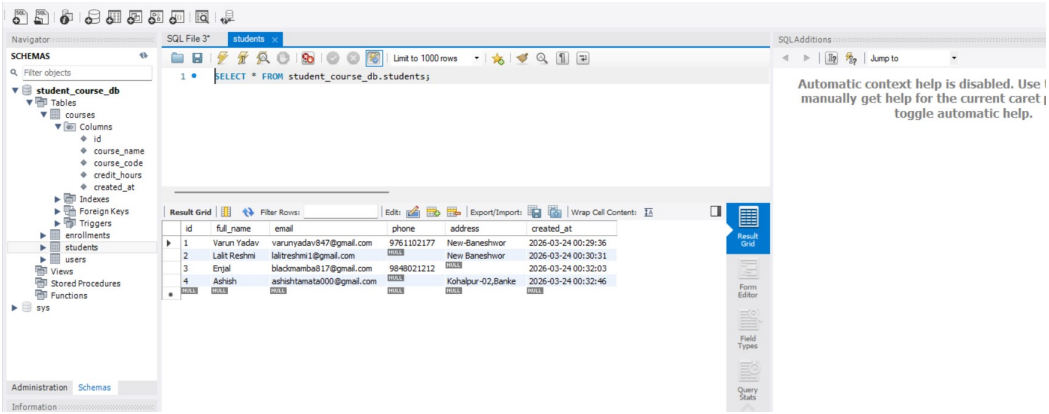


Enrollment database

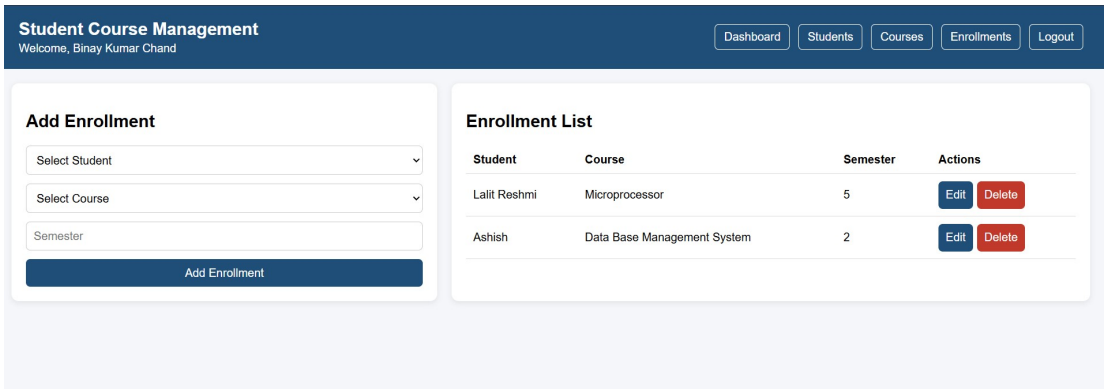
Edit Action



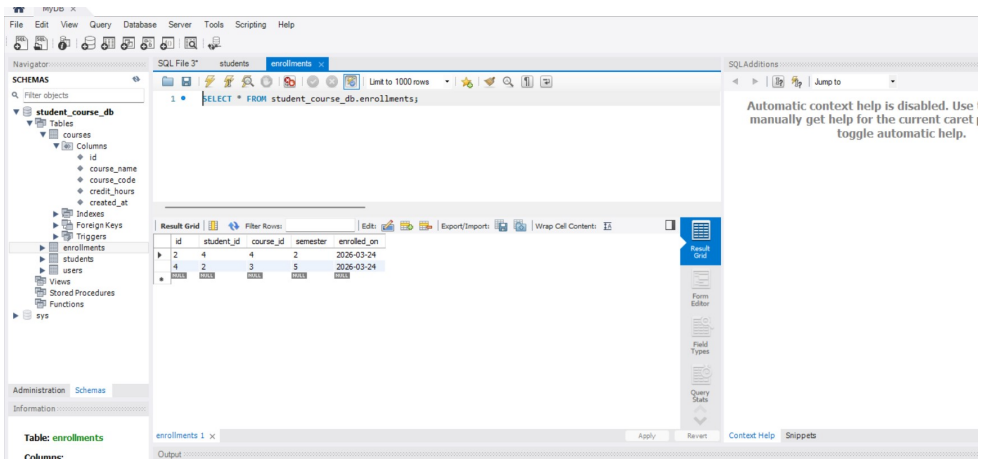
Edit Action in database



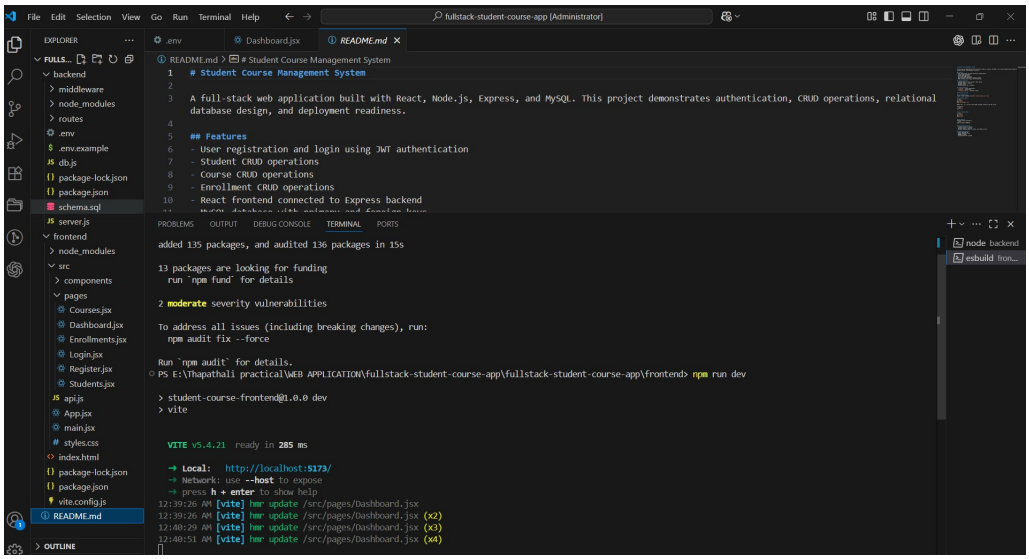
Delete Action



Delete Action in database



Terminal Output


File Edit Selection View Go Run Terminal Help
fullstack-student-course-app [Administrator]
EXPLORER
FULLS...
backend
middleware
node_modules
routes
.env
.env.example
db.js
package-lock.json
package.json
schema.sql
server.js
frontend
node_modules
src
components
pages
Courses.jsx
Dashboard.jsx
Enrollments.jsx
Login.jsx
Register.jsx
Students.jsx
api.js
App.jsx
main.jsx
styles.css
index.html
package-lock.json
package.json
vite.config.js
README.md
OUTLINE
.env
Dashboard.jsx
README.md
README.md > # Student Course Management System
Student Course Management System
A full-stack web application built with React, Node.js, Express, and MySQL. This project demonstrates authentication, CRUD operations, relational database design, and deployment readiness.
Features
- User registration and login using JWT authentication
- Student CRUD operations
- Course CRUD operations
- Enrollment CRUD operations
- React frontend connected to Express backend
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
added 135 packages, and audited 136 packages in 15s
13 packages are looking for funding
run `npm fund` for details
2 moderate severity vulnerabilities
To address all issues (including breaking changes), run:
npm audit fix --force
Run `npm audit` for details.
PS E:\Thapathali practical\WEB APPLICATION\fullstack-student-course-app\fullstack-student-course-app\frontend> npm run dev
> student-course-frontend@1.0.0 dev
> vite
VITE v5.4.21 ready in 285 ms
➜ Local: http://localhost:5173/
➜ Network: use --host to expose
➜ press h + enter to show help
12:39:26 AM [vite] hmr update /src/pages/Dashboard.jsx
12:39:26 AM [vite] hmr update /src/pages/Dashboard.jsx (x2)
12:40:29 AM [vite] hmr update /src/pages/Dashboard.jsx (x3)
12:40:51 AM [vite] hmr update /src/pages/Dashboard.jsx (x4)
node backend
esbuild fron...